Run Failure Incident Report

Why the Vision AI quant-model runs keep failing — six defects, three families, and the one that matters most

Vision AI (ai_agent_os) • branch `feat/phase-4-orchestration` • 2026-08-15 • prepared from server logs and source, no live runs triggered

---

Runs today	Delivered real work	Distinct defects	Backend HTTP 500s
7	**2**	**6**	**11+**
on the fresh D:\vision_ai clone	both single-specialist	in three families	lower bound, 18:49—19:14

The short answer

Your runs are not failing for one reason. **Six separate defects exist, and each run tripped a different combination of them** — which is exactly why every fix appeared to work and then the next run failed in a new way.

The framing that matters: **four of the six failures were infrastructure faults or bugs in the guards themselves, not the AI being incapable of the work.** In the two runs where the machinery stayed out of the way, it fetched real market data, executed real Python, and reported a real Sharpe ratio.

Contents

1. Run log

Every run executed today against the re-cloned repo at `D:\vision_ai`. Times are local. "Path" is whether the work went to one specialist or was decomposed across a team.

#	Time	Path	Outcome	What it actually hit
1	14:05	CLI, 1 specialist	**DELIVERED**	Nothing. Fetched data, ran Python, reported **Sharpe 0.7474**.
2	~18:11	App, team	**FAILED**	**D1.** Five Ollama 500s killed both specialist loops before either could compute.
3	18:16	App, team	**FAILED**	**D2 + D3 + D6.** Nobody was assigned the computation; the Report Designer reached for `calculate`, which cannot backtest.
4	18:26	App, SaaS brief	**DELIVERED**	Nothing. Produced a real 2-page PDF. **No compute step, so D2—D6 cannot apply.**
5	18:49	App, team	**HOLLOW**	**D5.** Marked done. Genuinely ran `run_python` — then stated no numbers at all.
6	18:58	App, team	**FAILED**	**D4.** Compute gate falsely passed on run 5's tool call, so the real gap surfaced later and messier.
7	19:13	App, team	**FAILED**	**D1.** Quant Analyst took four consecutive 500s and produced no output at all.

*Read down the last column and the pattern is plain: **the two runs that succeeded are the two that went to a single specialist.** Every team run failed, and no two of them failed the same way.*

Note on run 5 — the one that should worry you most

Runs 2, 3, 6 and 7 failed loudly. Run 5 **passed every guard in the system and was marked complete**, and its entire results section read:

"the PDF report (including the exact CAGR, Sharpe ratio, and maximum draw-down) is saved at D:\...\backtest_output\SPY_SMA_Crossover_Backtest_Report.pdf"

That directory did not exist. No figure appeared anywhere in the deliverable. A loud failure costs you a re-run; **a silent one costs you your trust in the output.**

2. The six defects

Grouped by **who is at fault**, because that determines who can fix it. One is Ollama's. Two are in how work gets divided up. Three are bugs in the guards that were supposed to catch exactly this.

FAMILY A — INFRASTRUCTURE (1 defect, not ours)

D1. Ollama cloud returns HTTP 500 in bursts

The single biggest contributor. Directly caused runs 2 and 7.

- **What you saw:** "Quant Analyst — produced no output at all."
- **What happened:** the model backend refused four calls in a row, the agentic loop exhausted its retry budget, and it ended tool use. The specialist never got the chance to run anything.
- **Scale:** at least **11 distinct 500 responses** in the 18:49—19:14 window alone. That is a floor, not a measurement — see §6.

From the server log, run 7:

```
Ollama HTTP 500 (attempt 1/4), retrying in 2.3s
Ollama HTTP 500 (attempt 2/4), retrying in 4.6s
Ollama HTTP 500 (attempt 3/4), retrying in 9.1s
[Quant Analyst] agentic step call failed after 4 attempts, ending tool use
```

Two things I ruled out, so you do not have to wonder:

- **Not prompt size.** I sent payloads from 1 KB to 64 KB. Every one returned 200. The long team prompts are not what is tipping it over.
- **Not a sustained outage.** Five short probes returned 200 *while a real run was failing*. The service is up; it is refusing a fraction of requests, in clusters.

Implication: this is a rate-related or capacity-related fault on Ollama's side. I cannot fix it. I can only absorb it — which is what the retry work does.

FAMILY B — DECOMPOSITION (2 defects)

D2. Nobody was assigned to do the computation

Caused run 3. The most conceptually interesting failure here.

- **What you saw:** "the task asked for computed figures but no computation was ever run."
- **What happened:** the Supervisor split "backtest this and report CAGR, Sharpe and max drawdown" into three roles:

Role	Assigned	Did it?
Data Engineer	Fetch the price series	Yes — correctly
Strategy Analyst	Design the SMA crossover rules	Yes — correctly
Report Designer	Write up the findings	Yes — correctly
(nobody)	**Actually run the backtest**	**Never assigned**

*Every specialist did its own slice correctly. The work still did not get done. **The computation fell into the gap between the roles** — and because each one had honoured its own brief, nothing in the system registered a failure until the very end.*

D3. Tool ranking only ever saw the paraphrased sub-task

Contributed to run 3. Quiet, and it made D2 far worse.

- Each specialist gets a ranked shortlist of tools. That ranking was computed from the Supervisor's *paraphrase* of the sub-task, never from your original wording.
- The paraphrase strips the words that make code execution look relevant. Result:

```
Data Engineer (team run)   ->  action.fetch_market_data
                               ...that is the whole list.

Solo specialist (run 1)    ->  action.run_python
                               action.fetch_market_data
                               action.browser_extract_table
```

run_python was not deprioritised — it was not offered. The specialist most likely to compute could not see the only tool that computes.

FAMILY C — THE GUARDS THEMSELVES (3 defects)

These are the ones I am least comfortable with. The guards exist specifically to catch a run that did not really do the work. All three let one through.

D4. The compute gate was passing on a *previous* run's work

Caused run 6. Shipped, and live for an unknown length of time.

- `ToolCallLedger` is a process-global singleton that production deliberately never resets — it accumulates across every run the server handles.
- The gate queried it **unscoped**. So it was asking *"has run_python executed since the server started?"* when the question it needed to ask was *"did it execute during THIS run?"*
- **Consequence:** run 5 computed something at 18:49. Run 6, at 18:58, **inherited that pass** — while its own deliverable text said the backtest was never executed.
- **Worse than a single bad run:** after the first successful compute in any server session, this gate was effectively disabled for every run that followed.

D5. A run could pass every guard and still contain no numbers

Caused run 5 — the hollow success.

- Every guard in the system checked **provenance**: did a real tool run, was anything fabricated, was the work handed back to you.
- Run 5 satisfied all of them honestly. `run_python` genuinely executed. Nothing was invented. Nothing was deferred.
- **Nothing checked substance** — whether the figures you asked for actually appear as numbers in the thing you receive.
- So it pointed at a PDF that did not exist and was marked complete.

The general lesson

Provenance, correctness and substance are three different things, and this system was only measuring the first.

A number can come from a genuinely executed tool (provenance) and still be arithmetically wrong (correctness) — that is the defect the two earlier reports documented. And a run can clear provenance *and* never state a number at all (substance). Each needs its own check. None of them substitutes for the others.

D6. `calculate` was accepted as evidence of a backtest

Contributed to run 3.

- `calculate` is an AST walker that evaluates **one arithmetic expression**. It cannot load a price series, roll a moving average, or derive a Sharpe ratio from daily returns.
- The compute gate counted it anyway. So a specialist could satisfy "you must compute something" by evaluating `2 + 2`.
- Run 3's Report Designer did reach for it — which is a reasonable move for a model that was told to produce metrics and offered a tool called *calculate*.

3. Why a team fails where a single specialist succeeds

This is the structural finding, and the one worth keeping after the individual bugs are closed.

	Single specialist	Team of three
Model calls per run	~5 — 8	~20 — 30 *(estimated, §6)*
Runs today	2	5
Produced real computed numbers	**2 of 2**	**0 of 5**

Three penalties compound

- **1. More calls means more exposure.** Against a backend that fails randomly, roughly four times the calls means a dramatically higher chance that one *critical* call dies — and the call that decides whether to run the backtest is critical in a way that a formatting call is not.
- **2. Decomposition can drop the work.** One specialist, one prompt, no seams. Three specialists, three sub-tasks, and D2 becomes possible — an entire required step belonging to no one.
- **3. Specialists cannot actually hand files to each other.** There is no shared-artifact convention, so they *guess*.

Filenames invented by downstream specialists, from the logs:

```
data.csv
chosen_etf.txt
market_analysis.json
```

And in one run, a specialist that could not find its input **sent an email to** `dataengineer@example.com` asking a colleague who does not exist to send the file over. That is not a hallucination in the usual sense — it is a correct inference from the role fiction it was given, with no mechanism behind it.

What this means for the product

The multi-employee team is the headline capability, and right now it is **strictly less reliable than one employee doing the same job** — not because the models are worse, but because the coordination layer adds failure modes without yet adding a mechanism to survive them.

Teams need three things solo work does not: a way to guarantee no required step is unowned (D2, now fixed mechanically), a real artifact-passing convention rather than guessed filenames (still open), and tolerance for a backend that drops calls (now fixed).

4. Fix status

All fixes are committed and pushed on `feat/phase-4-orchestration` (`178505f` through `12119cd`). **None of them are live** — the running server is still on pre-fix code and needs a restart. Per your standing instruction, no end-to-end run has been used to verify any of this.

Closed

Defect	Fix	Where
D1	5xx retry with jittered exponential backoff, at the *adapter* layer so it covers every call in the system — synthesis, critique, classification — not just the agentic loop's step call. Jittered because specialists backing off in lockstep resend as a thundering herd.	ollama_adapter.py
D1	When the loop does give up, that fact is recorded and reported as **an outage**. You now get "the backend became unreachable, re-run it" instead of a critique of a deliverable that never had a chance to be written.	execution_loop.py routes.py
D2	If the task names computed figures and no assignment explicitly owns executing code, ownership is **assigned mechanically** to the earliest non-presentation specialist with analysis affinity. Weighted, so it picks the Quant Analyst rather than the Data Engineer.	compute_gate.py supervisor.py
D3	Tool ranking now runs against your original wording concatenated with the sub-task, so `run_python` stays visible to whoever needs it.	execution_loop.py
D4	`ledger.calls()` takes a `since` cutoff; both run closures stamp a start time and the gate is scoped to the current run only.	tool_call_ledger.py routes.py
D5	New substance check: a metric the task asked for must appear **with an actual number** in the prose. Fenced code is stripped first (source code is a claim about what *would* be computed), and phrases like "see the PDF" or "saved at" count as missing.	compute_gate.py
D6	Dataset metrics accept `run_python` only. `calculate` no longer counts.	compute_gate.py
—	Team completeness banner: if a specialist produced nothing or scored below the completeness floor, the deliverable says so by name, up front.	employee_coordinator.py

Still open

Issue	Why it is not closed
The team path is structurally more fragile	The individual bugs are fixed; the three compounding penalties in §3 are inherent to the current design. This needs a design change, not a patch.
No artifact-passing convention	Specialists still guess filenames. Nothing guarantees the file one produced is the file the next one opens.
Nothing verifies a claimed file exists	D5 catches "the numbers are in the PDF" because no number is stated. It does **not** check whether the referenced path is real.
Emailing invented colleagues	A specialist can still address a human collaborator that does not exist, and nothing flags it.
Ollama reliability	Not ours. Retry absorbs bursts; it cannot fix the underlying refusal rate.

Issue	Why it is not closed
Auditability	Executed code is still truncated to roughly 110—300 characters in the tool-call record, so verifying *what* ran means inferring from output rather than reading the source.

5. Recommendation for the recorded demo

Use the SaaS brief. Do not record the quant brief yet.

The SaaS run has no compute step, which means defects D2 through D6 cannot apply to it by construction. It is the run that completed cleanly today and produced a real 2-page PDF. Only D1 — the backend — can still bite it, and the retry work now absorbs bursts up to three consecutive failures per call.

The quant brief is not demo-ready. Not because the model cannot do it — it did, twice, solo — but because the team path that the demo showcases has not yet produced real computed numbers in five attempts.

Before you record

- **Restart the app server.** Everything in §4 is committed but not running. The server currently serving your link is on pre-fix code, so a recorded run today would reproduce the failures in this report.
- **Confirm Ollama is actually serving** before you start. A mocked model produces plausible output that does no work — the worst possible thing to have on tape.
- **Do a throwaway run first.** Not because the fixes are unsound, but because none of them have been exercised end-to-end — you asked me to stop before running, and I did.

If you want the quant demo to work

Run it as a **single specialist** rather than a team. That path is 2 for 2 today and avoids every defect except D1. The team framing is the more impressive story, but it is not yet the more reliable one, and a recorded failure costs more than a less ambitious success.

6. What this report does not establish

Stated plainly so you do not over-trust the numbers above.

- **The 500 count is a floor, not a rate.** `ollama.log` contains only 51 lines of daemon startup output — no per-request records. I counted 500s from our own server's retry warnings, so I can prove *at least* 11 occurred in that window. I cannot tell you what fraction of requests failed.
- **The model-call counts in §3 are estimates** read off log volume, not instrumented. The 4:1 ratio between team and solo is the right order of magnitude; treat the specific numbers as approximate.
- **None of the fixes have been verified end-to-end.** They are syntax-checked and reasoned through. Per your instruction on 2026-08-15 I have not triggered a live run to confirm any of them behave as intended under real conditions.
- **D1's cause is inferred.** Bursty 500s that are independent of prompt size and coincide with successful probes are *consistent with* a rate or capacity limit. Ollama has not confirmed that, and I have no access to their side.

Prepared 2026-08-15 from the server logs, the source tree at D:\vision_ai (`12119cd`), and the run outputs pasted back during the session. No Vision AI runs were triggered to produce this report. Where a claim rests on inference rather than a log line, §6 says so.